COMPUTER SCIENCE + COMPUTER ENGINEERING

A guided undergraduate-to-graduate route for games

04

Gate C: Native core and language runtime

Project specification and review rubric

Integrate deterministic simulation, ownership, algorithms, a compiler, a bounded VM and the C bridge.

PACKAGE 04 / EXPANDED VOLUME 03 / SEPTEMBER 28, 2026
Independent study, implementation, evidence and review.

Contents

The artifact to build 3
Written specification before implementation 3
Stage and resource contracts 4
Milestones and acceptance evidence 4
Required adversarial and boundary matrix 6
Submission dossier 7
Review rubric and non-negotiable checks 7
Oral or recorded defense 7
Continuity with the original program 8
Primary reference directory 8
Reading by topic 10

Gate C: Native simulation and bounded language runtime

The artifact to build

Create a student-owned repository containing a deterministic command-line simulation, explicit resource ownership, a small educational container/algorithm comparison, and an expression language implemented as lexer, parser, AST evaluator, bytecode compiler and checked VM. Add an error-preserving constant folder and a finite-state recognizer. Complete the C bridge for the hardware/firmware direction.

This checkpoint is the integration of the five module projects. It is not a request to build a commercial game engine, a complete optimizing compiler, a secure modding platform or a real hardware driver. Terminal output is sufficient. The finished reference demonstrations in this bundle are component examples, not a finished Gate C submission.

Written specification before implementation

Write a two-to-four-page contract. State the simulation's initial state, command format, tick policy, movement/collision rules, numeric domains, entity identity, invalid-input policy and output format. Identify which behavior is inherited from your earlier game and which is a deliberate change. Choose an integer deterministic baseline before attempting floating-point or multithreaded replay.

For the language, implement the following base grammar; whitespace separates tokens but does not change precedence. Offsets are zero-based byte positions in the source. The implementation recognizes ASCII digits and the listed punctuation, not locale-dependent numeric spellings.

```
expression := term { ("+" | "-") term }
term       := unary { ("*" | "/") unary }
unary      := "-" unary | primary
primary    := integer | "(" expression ")"
integer    := digit { digit }
digit      := "0" | "1" | ... | "9"
```

Addition, subtraction, multiplication and division use integer values in **[-1,000,000,000, +1,000,000,000]**. Decimal literals are nonnegative and must fit that positive bound; a negative source value uses unary minus. Division truncates toward zero. Check every operation; division by zero and results outside the domain are errors. A leading plus sign, identifiers, decimals, scientific notation, implicit multiplication and trailing tokens are rejected in the baseline. Leading zeros are accepted. These are deliberate small-language choices, not claims about every programming language.

Operations can use int64_t intermediates because bounded multiplication is at most 10^18 in magnitude. Validate operands before relying on that bound. Do not extend the value domain without re-deriving intermediate overflow safety. The expression `-7/3` yields -2. The source `2+3*4` yields14 and `(2+3)*4` yields20. The source `8-3-2` yields3 because subtraction is left-associative.

The type-checker exercise in15.1 separately demonstrates Number/Boolean rules. It does not imply that Boolean literals, variables or conditionals exist in this arithmetic grammar. Adding them is an optional extension requiring a new versioned specification and new tests.

Stage and resource contracts

Stage	Required boundary	Reference default
Source	Check bytes before scanning	16,384 bytes
Lexer	Count emitted tokens, including End	4,096 tokens
Parser	Count owned AST nodes	4,096 nodes
Parser	Bound nesting and structural AST depth	64
Compiler	Check emitted bytecode length	4,096 instructions
VM	Check every stack growth	256 values
VM	Charge before executing each instruction	4,096 fuel units
Arithmetic	Validate inputs and results	absolute value <=10^9

Lower these limits in tests so each boundary is observable with small fixtures. Test at the boundary and one past it. Structural depth and syntactic parenthesis nesting are different checks. A long flat subtraction chain can form a deep AST even when parentheses are shallow. Allocation failure is distinct from hitting an explicit course budget.

Only parser-produced, internally owned ASTs are trusted inputs to the tree evaluator and compiler in the reference design. Arbitrary externally supplied graphs could contain invalid indices, sharing or cycles; accepting those would require a separate validator and graph budget. Bytecode is checked at runtime for supported opcodes, underflow, stack growth, operand ranges, instruction length, fuel and final result count. This restricted system has no jumps, host callbacks or I/O and is not a general security boundary. [R14; P4-06]

Milestones and acceptance evidence

C1 — Reproducible native core

Build a library target for simulation rules, a command-line demonstration and an automated test target. Keep a header/implementation split. Produce at least three replay fixtures: ordinary behavior, boundaries/no-op, and invalid input. Compare the native results with the earlier implementation when their contracts match. Preserve complete state at each step, not only a final screenshot.

Accept when: a new build directory produces the same specified outputs; warnings are clean under the chosen warning policy; tests run in Debug and Release; integer and ownership differences from the earlier implementation are explained. Source files, generated build products and executable selection are unambiguous.

C2 — Owned resources and stable identity

Use the Module12 resource manager. Draw ownership and borrowing relationships. Implement resource acquisition, release, move and replacement; inject failure after at least one resource has actually been acquired. Show that replacement failure preserves the previous valid object where the strong guarantee is claimed.

Accept when: isolated tests end with zero live mock resources; successful acquisitions and releases match; no accidental owner copying occurs; moved-from and self-move policies are explicit; references retained across invalidating mutations are eliminated or validated using an explicit handle policy.

C3 — Data structures and algorithms with evidence

Include the four educational structures from Module13 with representation invariants. Compare at least two structures or layouts on equivalent representative workloads. Include the pathfinding visualizer and asset dependency planner from Module14, or a comparably demanding integration approved by the reviewer.

Accept when: golden traces and seeded differential tests agree; path edges and total costs are independently checked; cyclic dependencies and unreachable destinations are rejected explicitly; shortest-path claims state the heuristic and cost assumptions; cost reports distinguish expected, amortized and worst-case behavior. Timing results include raw samples and correctness checks.

C4 — Frontend and tree evaluation

Build the lexer, located tokens, recursive-descent parser and owned AST. Provide a way to print or inspect the AST. Consume all source tokens. Reject malformed input before execution. Associate runtime arithmetic errors with the operator offset.

Accept when: precedence, associativity, unary minus, parentheses and truncating division match the specification. Empty input, unknown names, unexpected characters, missing delimiters and trailing tokens fail deliberately. Source/token/node/depth limits are enforced before uncontrolled growth. Use independent golden values as well as generated expressions.

C5 — Bytecode and checked execution

Emit postorder stack instructions. Define opcode, operand and source-offset fields. Check every stack operation and final stack depth. Compile/evaluate the same source through both execution paths. Add a finite-state recognizer with a complete transition table, and constant-fold only operations whose checked evaluation succeeds.

Accept when: tree/VM results and relevant error categories/offsets agree. Malformed bytecode cannot pop an empty stack, exceed stack or fuel limits, or silently accept an unknown opcode. `8 3 SUB` produces5, not-5. `(1/0)*0` retains its division error. A fully valid constant expression may fold to one PUSH. Shared helpers are identified so that differential tests are not falsely called independent arithmetic proofs.

C6 — C bridge and reproducible review

Compile at least one translation unit as C17, not as C++. Use a C-compatible declaration header, an opaque create/destroy interface, a C++ owner wrapper, explicit field serialization and checked status returns. Use the four bridge exercises as evidence. Submit the review dossier and explain a changed case without following the supplied reference code.

Accept when: the verbose build shows actual C compilation; each successful allocation has the matching release; no C++ exception crosses the C API; native structure offsets are observed rather than assumed; an eight-byte wire format is tested field by field; failures preserve the documented output policy.

Required adversarial and boundary matrix

Area	Cases to include	Required observation
Simulation	empty/invalid command, edge, no-op, large legal tick	specified status and unchanged state on rejection
Ownership	partial construction, failed replacement, move assignment, destruction	unique ownership and balanced releases
Vector-facing API	growth, erase, missing ID, reused ID policy	no invalid retained borrow
Containers	empty/full, collision, duplicate, capacity growth, equal priority	invariants and return values preserved
Graphs	disconnected, cycle, zero cost, invalid ID, bad heuristic	correct rejection or appropriately limited claim
Syntax	empty, bad character, missing operand/parenthesis, unknown name, trailing token	located parse/lex error
Arithmetic	zero division, positive/negative bound, out-of-domain intermediate	error before unsafe native arithmetic
VM	invalid opcode, underflow, leftover values, stack limit, fuel limit	controlled error, not partial success
Optimizer	ordinary constants, failing subtree, noncommutative operations	equivalent specified value/error behavior
C boundary	null output, allocation failure, truncated bytes, version mismatch	status and documented unchanged/ null output

In addition, include five learner-designed edge/failure cases per module that are not copied from the bundle. Label why each is new, the predicted result, the observed result and the defect it could expose. This requirement measures transfer, not the ability to rename an existing fixture.

Submission dossier

Provide `README.md` with clean build/test/demo commands; a toolchain/configuration record; the simulation and language specifications; source and tests; replay and malformed-input fixtures; ownership diagram; budget/complexity table; benchmark raw samples; C bridge results; and a short known-limitations/provenance report. Include the output of a fresh-clone or fresh-extraction build without untracked local dependencies.

Your AI-help log should identify generated sections, the changes you made, the independent checks you ran and what you can explain without assistance. Referencing a library is acceptable; replacing the central parser, VM, educational structures or reasoning with a hidden library call is not equivalent evidence unless an explicit comparative assignment was agreed in advance.

Review rubric and non-negotiable checks

Dimension	Points	Evidence
Correctness and requirements	30	behavior matches the written operating envelope
Tests and verification	25	boundaries, injected failures, invariants and independent checks
Reasoning and design	15	ownership, grammar, algorithm assumptions and tradeoffs
Quantitative analysis	10	bounded costs and fair, uncertainty-aware comparisons
Reproducibility	10	clean commands, fixed fixtures, recorded environment
Communication and provenance	10	understandable contracts, honest limitations, assistance log

Suggested formative pass: **80/100**, all essential practical checks complete, and no unresolved critical defect within the claimed operating envelope. A polished visualizer cannot compensate for a dangling reference, double release, false optimality claim or unchecked bytecode operation. A performance improvement is not required; a defensible negative result is acceptable.

A successful sanitized run is useful evidence but is not proof of memory safety across unexecuted inputs. A reviewer may accept different technically sound implementations when their assumptions, behavior and evidence are explicit. [P4-03; P4-04]

Oral or recorded defense

Explain a pointer/lifetime defect and repair it without executing undefined behavior. Justify a structure choice under changed workloads. Extend the grammar on paper with one new operator, stating precedence, associativity, type behavior and tests. Classify a compile error, a link error and a runtime rejection. Explain why A* can return a wrong result with an invalid heuristic. Trace noncommutative bytecode. Finally, identify one claim your tests do not establish.

Use a new example supplied by the reviewer. Reading the answer key aloud does not demonstrate transfer. Keep the review constructive: record the gap, repair it with a changed exercise and retest before advancing to the next systems volume.

Sources and provenance

Continuity with the original program

This expansion was prepared for the September28,2026 curriculum edition. The original program manifest identifies Volume03 as C++, Algorithms and Language Implementation, Modules11–15, GateC and400–650 planning hours. Its lesson IDs, titles, outcomes, prerequisites, practical assignments and assigned readings are preserved in `source/ORIGINAL_VOLUME_03_MANIFEST.json`. The 60 original quiz questions and answers were recovered from the original `CURRICULUM.html`; the expansion adds40 application questions without replacing the originals.

Teaching explanations, worked examples, code, additional exercises, reviews, bridge material and project elaborations are original instructional synthesis for this bundle. Primary reading references support the underlying concepts. No external textbook, paper, compiler distribution, proprietary SDK or font file is included. Linked resources retain their own terms. Current documentation was consulted on September28,2026; historical course archives are intentionally used for durable foundations. Not every section of every historical source was re-reviewed in full.

The original assignments are preserved even where the starter supplies only a first contract. The full lab remains a learner requirement. Complete reference examples provide comparison evidence, not pre-completed module projects. The source JSON distinguishes original questions from additions.

Primary reference directory

R06 — 6.042J Mathematics for Computer Science

MIT OpenCourseWare. Proofs, graphs, counting and discrete probability.

https://ocw.mit.edu/courses/6-042j-mathematics-for-computer-science-spring-2015/

R11 — C++ Core Guidelines

C++ Core Guidelines editors. Resource safety, interfaces, classes and concurrency; guidance rather than the normative language standard.

https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

R12 — CMake tutorial

Kitware. Build systems; use documentation matching the installed release.

https://cmake.org/cmake/help/latest/guide/tutorial/index.html

R13 — 6.006 Introduction to Algorithms

MIT OpenCourseWare. Data structures, graph algorithms and dynamic programming.

https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/

R14 — Kaleidoscope and LLVM tutorials

LLVM Project. Compiler frontends, intermediate representation and JIT compilation.

https://llvm.org/docs/tutorial/

R15 — 6.004 Computation Structures

MIT OpenCourseWare. Logic, processor design, memory and systems.

https://ocw.mit.edu/courses/6-004-computation-structures-spring-2017/

R16 — Operating Systems: Three Easy Pieces

Remzi and Andrea Arpaci-Dusseau. Virtualization, concurrency and persistence; book by its authors.

https://pages.cs.wisc.edu/~remzi/OSTEP/

R20 — 15-466/666 Computer Game Programming

Carnegie Mellon University. Runtime systems and game-production programming.

https://graphics.cs.cmu.edu/courses/15-466/

P4-01 — CMake compile features and language standards

Kitware. Target requirements, language standards and extensions. Use the documentation matching your CMake release.

https://cmake.org/cmake/help/latest/manual/cmake-compile-features.7.html

P4-02 — What Every Computer Scientist Should Know About Floating-Point Arithmetic

David Goldberg / Oracle documentation. Author paper reproduced in official numerical-computation documentation; rounding and non-associativity background.

https://docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html

P4-03 — AddressSanitizer

LLVM / Clang. Compiler/runtime setup and kinds of executed memory errors detected. Detection is not an all-input proof.

https://clang.llvm.org/docs/AddressSanitizer.html

P4-04 — UndefinedBehaviorSanitizer

LLVM / Clang. Runtime instrumentation and supported checks. Platform/toolchain support must be verified.

https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html

P4-05 — Language standards supported by GCC

GNU Compiler Collection. C/C++ standards and language modes. This course selects C17/C++20, not a newest-standard claim.

https://gcc.gnu.org/onlinedocs/gcc/Standards.html

P4-06 — Kaleidoscope: implementing a parser and AST

LLVM Project. Frontend architecture reading; the course language and integer semantics are independently specified and do not require LLVM libraries.

https://llvm.org/docs/tutorial/MyFirstLanguageFrontend/LangImpl02.html

P4-07 — Fundamental types [basic.fundamental]

C++ working-draft editors / eel.is rendering. Current working-draft view, not the fixed published C++20 edition. Consult only the stable integer/representation concepts used here; later additions are outside this course baseline.

https://eel.is/c++draft/basic.fundamental

P4-08 — Vector modifiers, capacity and unordered-container requirements

C++ working-draft editors / eel.is rendering. Companion sections: https://eel.is/c++draft/vector.capacity and https://eel.is/c++draft/unord.req. Used for operation-specific invalidation rules; current draft contains later features not used by the course.

https://eel.is/c++draft/vector.modifiers

Reading by topic

Use R11 for ownership, interfaces, special members, exception safety and source-file organization; R12/P4-01 for builds; R13 for arrays, hashing, heaps, sorting, graphs and dynamic programming; R14/P4-06 for language frontends and compiler structure. R06 supports discrete proof and computability reasoning, and R15 connects representations to later hardware study. R16/R20 are cross-volume reading pointers, not dependencies of the supplied build.

Read the exact assignment printed under each lesson. Record the section actually studied and at least one exercise or derivation in the journal. The narrow course implementation deliberately differs from some reading examples: bounded integers instead of general floating-point language values, a small straight-line stack machine instead of an LLVM JIT, and mock resources instead of a graphics API. Do not merge incompatible assumptions silently.